

ENTRADAS DIGITALES — Laboratorio 02

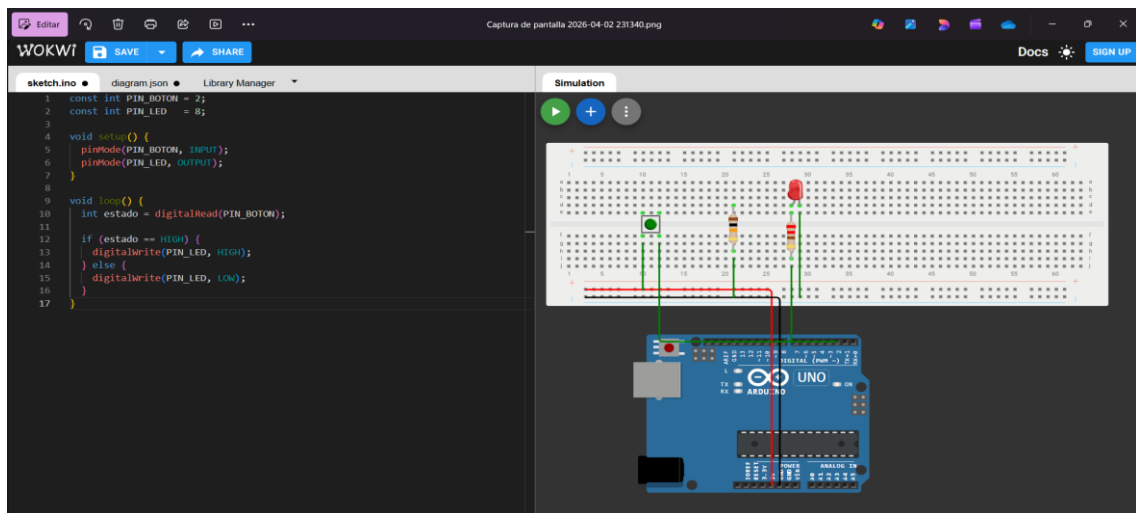
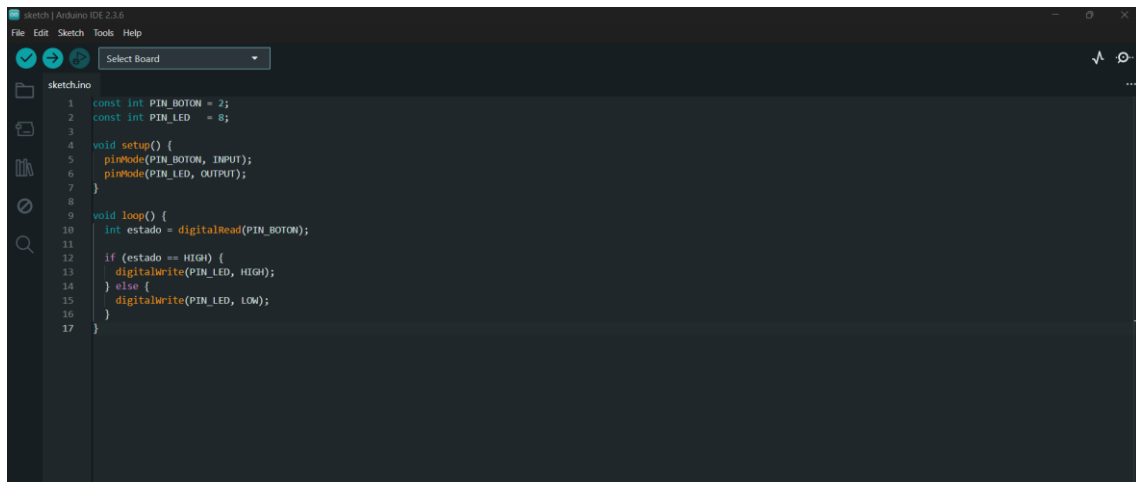
En esta primera parte se definieron los objetivos del laboratorio: aprender a configurar pines como entradas digitales, usar resistencias pull-up y pull-down, comprender el rebote mecánico y aplicar debounce. Se explicó que un pin flotante puede dar lecturas erráticas y que las resistencias pull-up/pull-down aseguran un nivel lógico estable.

Paso 1: Lectura básica con Pull-down externo

Se realizó la primera práctica conectando un pulsador con una resistencia de 10kΩ a tierra (pull-down). El LED se encendía al presionar el botón y se apagaba al soltarlo. El código configuró el pin como entrada normal (INPUT) y se usó `digitalRead()` para leer el estado. Además, se imprimió en el Monitor Serial el mensaje “Botón: PRESIONADO”.

Se comprobó cómo una resistencia pull-down evita lecturas falsas en un pin flotante.

Evidencias

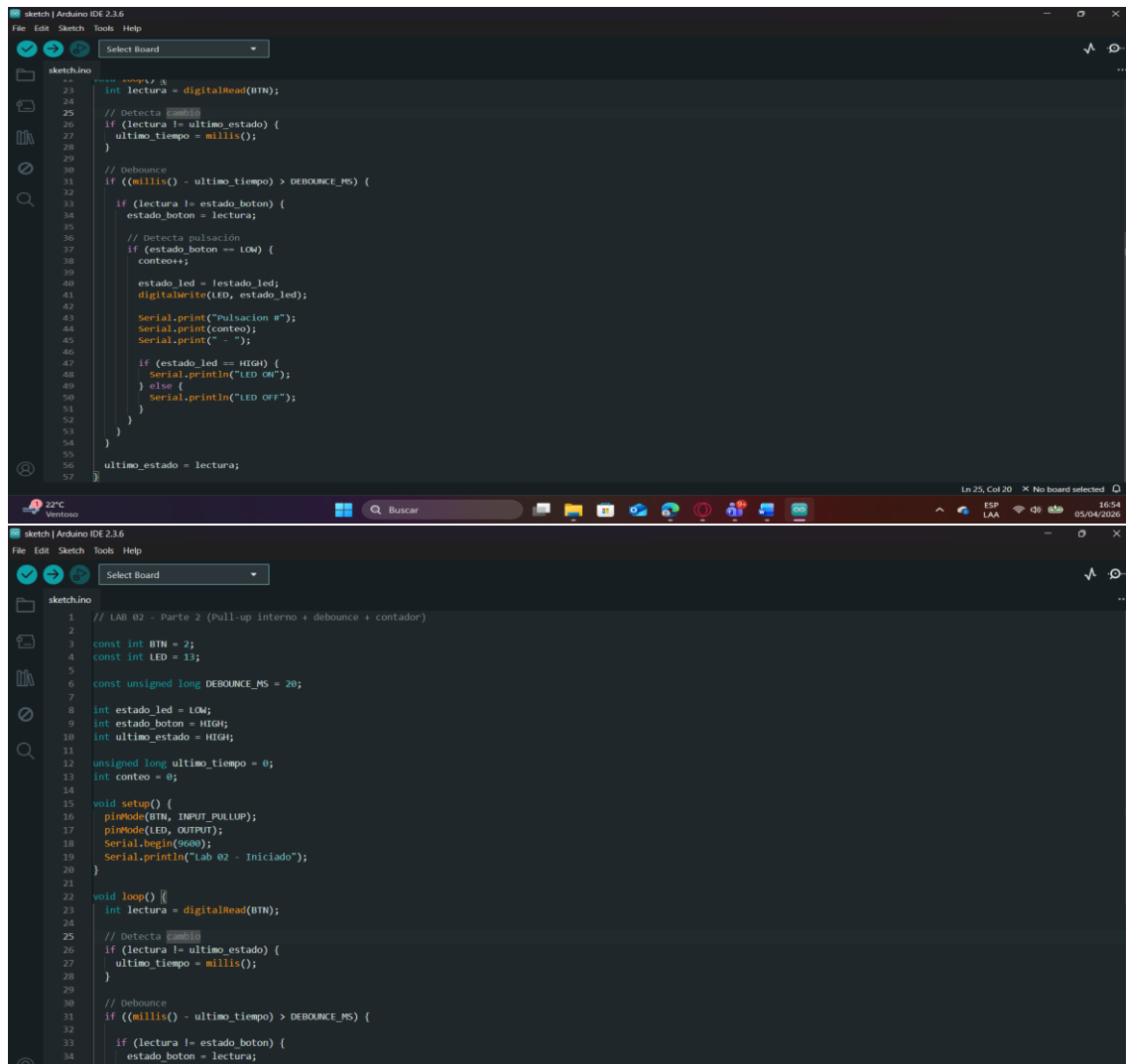


Paso 2: Pull-up interno y debounce

En este paso se activó el pull-up interno del microcontrolador (INPUT_PULLUP), lo que invierte la lógica: reposo = HIGH, presionado = LOW. Se implementó un algoritmo de debounce usando millis() para filtrar los rebotes del pulsador sin bloquear el procesador. El LED alternaba su estado en cada pulsación válida y se llevaba un conteo en el Monitor Serial.

Se entendió la importancia de usar técnicas no bloqueantes en sistemas embebidos, evitando delay () para que el procesador pueda seguir ejecutando otras tareas.

Evidencias



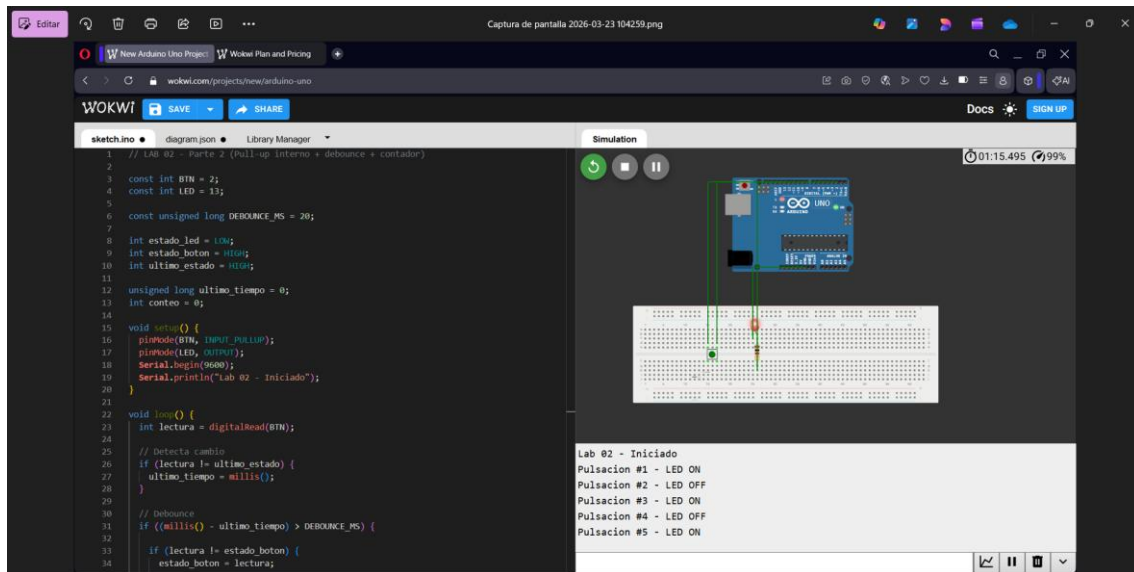
The image displays two screenshots of the Arduino IDE 2.3.6 interface, showing the code for the Pull-up internal and debounce step.

The top screenshot shows the code for the Pull-up internal and debounce step. The code is as follows:

```
1 // LAB 02 - Parte 2 (Pull-up interno + debounce + contador)
2
3 const int BTN = 2;
4 const int LED = 13;
5
6 const unsigned long DEBOUNCE_MS = 20;
7
8 int estado_led = LOW;
9 int estado_boton = HIGH;
10 int ultimo_estado = HIGH;
11
12 unsigned long ultimo_tiempo = 0;
13 int conteo = 0;
14
15 void setup() {
16   pinMode(BTN, INPUT_PULLUP);
17   pinMode(LED, OUTPUT);
18   Serial.begin(9600);
19   Serial.println("Lab 02 - Iniciado");
20 }
21
22 void loop() {
23   int lectura = digitalRead(BTN);
24
25   // Detecta cambio
26   if (lectura != ultimo_estado) {
27     ultimo_tiempo = millis();
28   }
29
30   // Debounce
31   if ((millis() - ultimo_tiempo) > DEBOUNCE_MS) {
32
33     if (lectura != estado_boton) {
34       estado_boton = lectura;
35
36       // Detecta pulsación
37       if (estado_boton == LOW) {
38         conteo++;
39
40         estado_led = !estado_led;
41         digitalWrite(LED, estado_led);
42
43         Serial.print("Pulsacion ");
44         Serial.print(conteo);
45         Serial.print(" - ");
46
47         if (estado_led == HIGH) {
48           Serial.println("LED ON");
49         } else {
50           Serial.println("LED OFF");
51         }
52       }
53     }
54   }
55   ultimo_estado = lectura;
56 }
57
```

The bottom screenshot shows the code for the Pull-up internal and debounce step. The code is as follows:

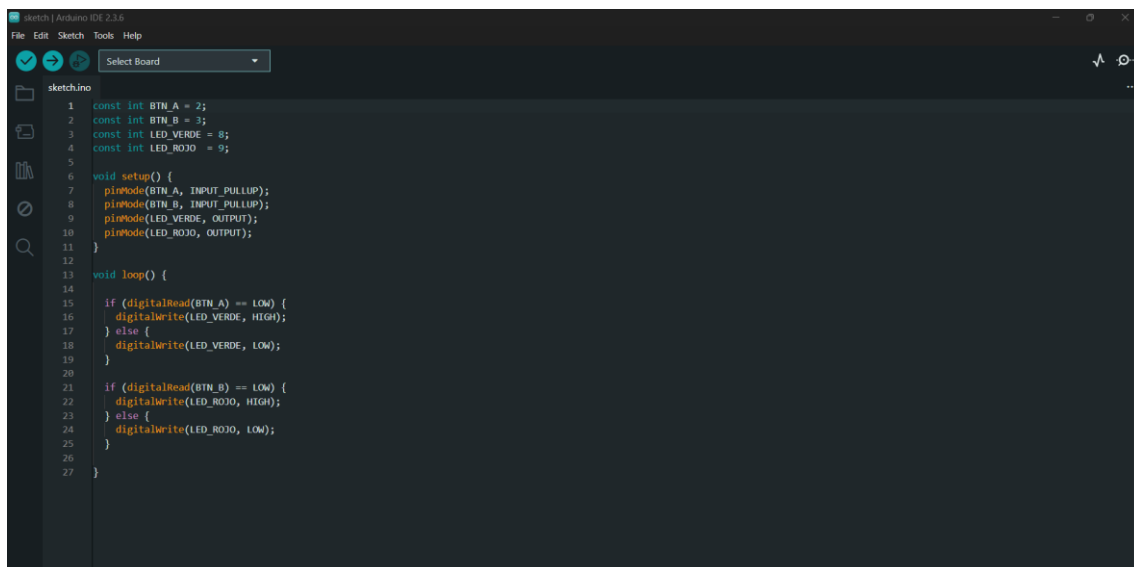
```
1 // LAB 02 - Parte 2 (Pull-up interno + debounce + contador)
2
3 const int BTN = 2;
4 const int LED = 13;
5
6 const unsigned long DEBOUNCE_MS = 20;
7
8 int estado_led = LOW;
9 int estado_boton = HIGH;
10 int ultimo_estado = HIGH;
11
12 unsigned long ultimo_tiempo = 0;
13 int conteo = 0;
14
15 void setup() {
16   pinMode(BTN, INPUT_PULLUP);
17   pinMode(LED, OUTPUT);
18   Serial.begin(9600);
19   Serial.println("Lab 02 - Iniciado");
20 }
21
22 void loop() {
23   int lectura = digitalRead(BTN);
24
25   // Detecta cambio
26   if (lectura != ultimo_estado) {
27     ultimo_tiempo = millis();
28   }
29
30   // Debounce
31   if ((millis() - ultimo_tiempo) > DEBOUNCE_MS) {
32
33     if (lectura != estado_boton) {
34       estado_boton = lectura;
35
36       // Detecta pulsación
37       if (estado_boton == LOW) {
38         conteo++;
39
40         estado_led = !estado_led;
41         digitalWrite(LED, estado_led);
42
43         Serial.print("Pulsacion ");
44         Serial.print(conteo);
45         Serial.print(" - ");
46
47         if (estado_led == HIGH) {
48           Serial.println("LED ON");
49         } else {
50           Serial.println("LED OFF");
51         }
52       }
53     }
54   }
55   ultimo_estado = lectura;
56 }
57
```

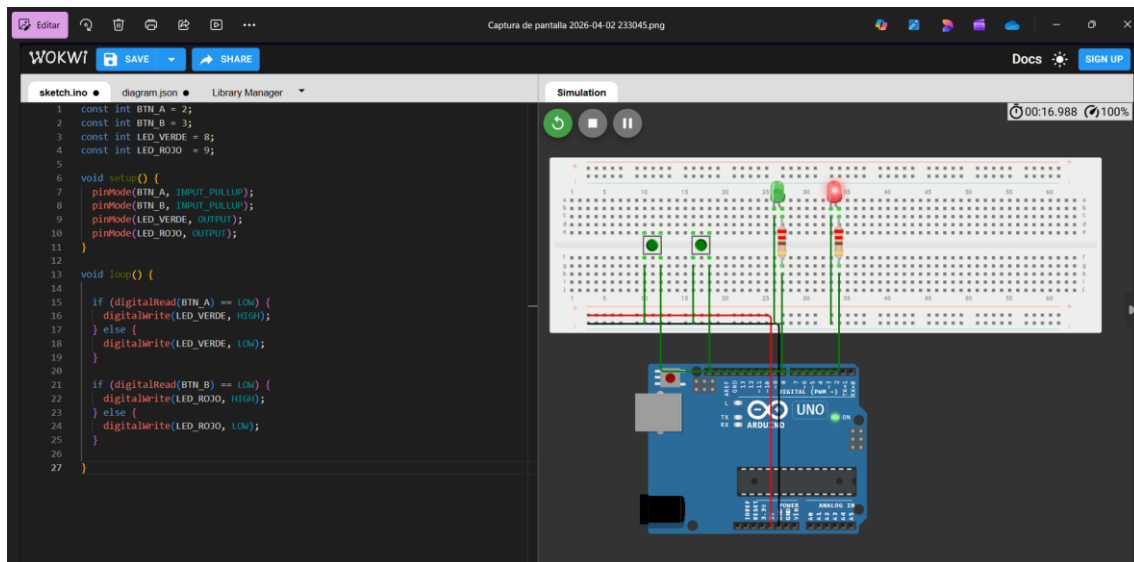
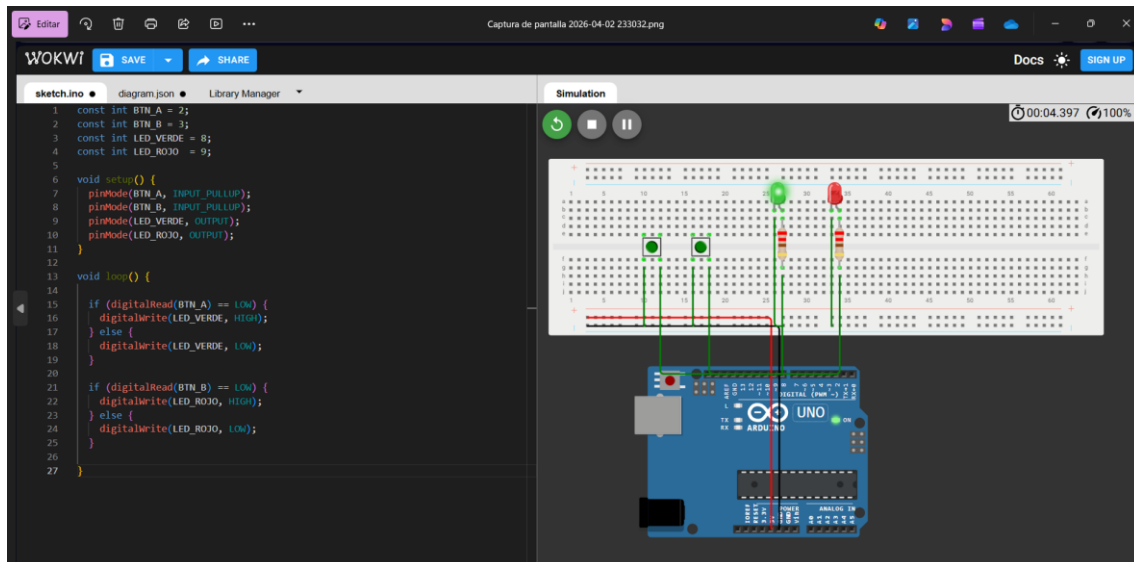


Paso 3: Ejercicios prácticos

Finalmente, se llenó la tabla de verificación para comprobar que cada criterio del laboratorio se cumplía: funcionamiento con pull-down, lógica invertida con pull-up, eliminación del rebote, conteo correcto y parpadeo según el número de pulsaciones.

Evidencias





Conclusiones

En este laboratorio se logró comprender cómo funcionan las entradas digitales en un microcontrolador y la importancia de las resistencias pull-up y pull-down para evitar lecturas erráticas. Se verificó que el rebote mecánico de los pulsadores puede generar múltiples lecturas falsas y que el uso de `millis()` permite implementar un debounce eficiente sin bloquear el procesador. Los ejercicios adicionales reforzaron el aprendizaje práctico, mostrando cómo ampliar la funcionalidad con más pulsadores y simulaciones en Wokwi. En conclusión, se adquirió experiencia fundamental para el diseño de sistemas embebidos confiables y eficientes.